Nginx Deployment Lab Manual - Two Approaches

This lab teaches two methods of deploying a simple website using Nginx.

Approach 1: Use the official Nginx Docker image.
Approach 2: Install Nginx manually inside an Ubuntu Linux container.

Each exercise contains:
• Objective
• Command
• Explanation
• Expected Output
• Screenshot Placeholder

Approach 1 - Pull Nginx Image

Command:

docker pull nginx

Explanation:

Downloads the official Nginx image from Docker Hub.

Expected Output:

```
laxmanp@W4N00TW13C-laxmanp Linux1 % docker pull nginx
Using default tag: latest
latest: Pulling from library/nginx
78e1a52d6c50: Pull complete
97a97a349235: Pull complete
572aad3b769d: Pull complete
8ae94b355a63: Pull complete
0ef4315fa7b7: Pull complete
43899da09563: Pull complete
3c03b4506f3b: Download complete
37647833f672: Download complete
Digest: sha256:42f2d24ae18df9b5251d1cc45548085656d2335e9338fd150a24e415462d151f
Status: Downloaded newer image for nginx:latest
docker.io/library/nginx:latest

What's next:
    View a summary of image vulnerabilities and recommendations → docker scout quickview nginx
laxmanp@W4N00TW13C-laxmanp Linux1 %
```

Approach 1 - Run Nginx Container

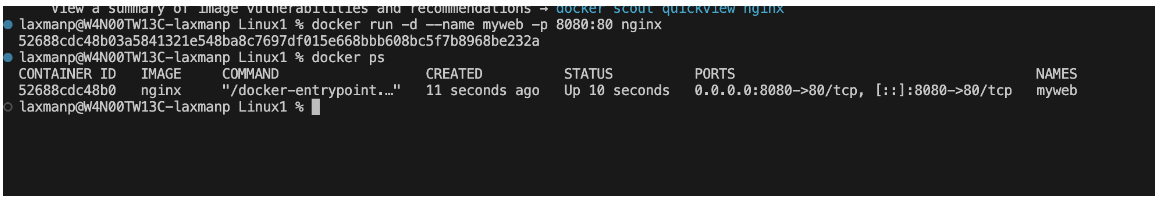
Command:

docker run -d --name myweb -p 8080:80 nginx

Explanation:

Starts an Nginx container and maps port 8080 on the host to port 80 inside the container.

Expected Output:


```
    View a summary of image vulnerabilities and recommendations → docker scout quickview nginx
laxmanp@W4N00TW13C-laxmanp Linux1 % docker run -d --name myweb -p 8080:80 nginx
52688cdc48b03a5841321e548ba8c7697df015e668bbb608bc5f7b8968be232a
laxmanp@W4N00TW13C-laxmanp Linux1 % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                     NAMES
52688cdc48b0   nginx     "/docker-entrypoint.…"   11 seconds ago   Up 10 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   myweb
laxmanp@W4N00TW13C-laxmanp Linux1 %
```

Approach 1 - Verify Running Container

Command:

docker ps

Explanation:

Displays running containers.

Expected Output:

```
52688cdc48b03a5841321e548ba8c7697df015e668bbb608bc5f7b8968be232a
laxmanp@W4N00TW13C-laxmanp Linux1 % docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                   NAMES
52688cdc48b0   nginx     "/docker-entrypoint.…"   11 seconds ago   Up 10 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   myweb
laxmanp@W4N00TW13C-laxmanp Linux1 %
```

Approach 1 - Create Custom index.html

Command:

Create a local index.html file.

Explanation:

Creates a custom web page.

Expected Output:

```html
<!-- Sample HTML for demonstrating NGINX web server with Docker on Linux -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Welcome - Our Landing Page</title>

    <!-- Internal CSS Styles -->
    <style>
        /* Reset and Base Styles */
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            line-height: 1.6;
```

Approach 1 - Copy File

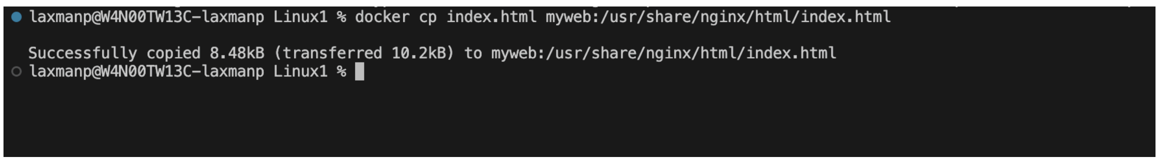
Command:

docker cp index.html myweb:/usr/share/nginx/html/index.html

Explanation:

Copies the local HTML file into the Nginx web root.

Expected Output:



Approach 1 - Verify Website

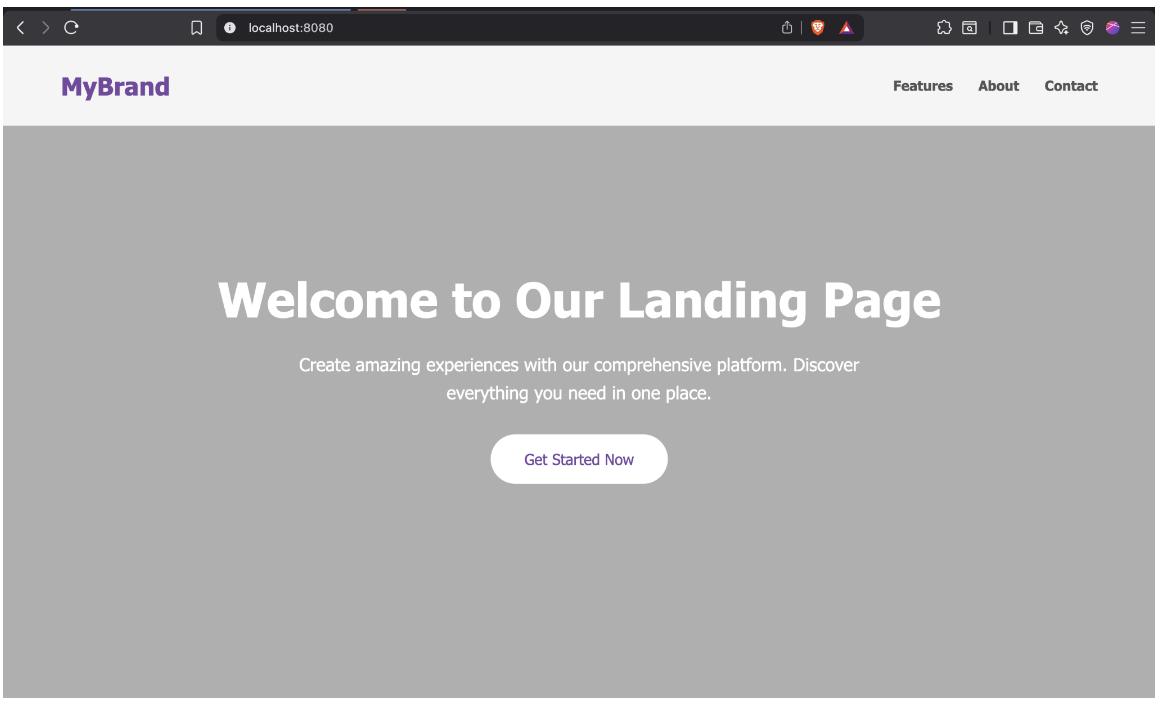
Command:

Browse to http://localhost:8080

Explanation:

Displays the custom web page.

Expected Output:



Approach 2 - Pull Ubuntu

Command:

docker pull ubuntu:24.04

Explanation:

Downloads the Ubuntu Linux image.

Expected Output:

```
laxmanp@W4N00TW13C-laxmanp Linux1 % docker pull ubuntu:24.04

24.04: Pulling from library/ubuntu
Digest: sha256:786a8b558f7be160c6c8c4a54f9a57274f3b4fb1491cf65146521ae77ff1dc54
Status: Image is up to date for ubuntu:24.04
docker.io/library/ubuntu:24.04

What's next:
    View a summary of image vulnerabilities and recommendations → docker scout quickview ubuntu:24.04
laxmanp@W4N00TW13C-laxmanp Linux1 %
```

Approach 2 - Start Ubuntu

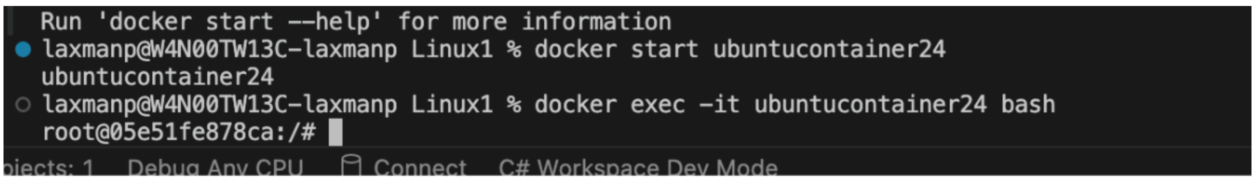
Command:

docker run -it --name ubuntu-nginx ubuntu:24.04 bash

Explanation:

Starts an interactive Ubuntu container.

Expected Output:



Approach 2 - Update Package Repository

Command:

apt update

Explanation:

Downloads the latest package metadata.

Expected Output:

```
laxmanp@W4N00TW13C-laxmanp Linux1 % docker exec -it ubuntucontainer24 bash
root@05e51fe878ca:/# apt update
Hit:1 http://ports.ubuntu.com/ubuntu-ports noble InRelease
Hit:2 http://ports.ubuntu.com/ubuntu-ports noble-updates InRelease
Hit:3 http://ports.ubuntu.com/ubuntu-ports noble-backports InRelease
Hit:4 http://ports.ubuntu.com/ubuntu-ports noble-security InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
5 packages can be upgraded. Run 'apt list --upgradable' to see them.
root@05e51fe878ca:/#
```

Approach 2 - Install Nginx

Command:

apt install nginx -y

Explanation:

Installs the Nginx web server.

Expected Output:

```
inux-gnu/perl/5.38.2 /usr/local/share/perl/5.38.2 /usr/lib/aarch64-linux-gnu/perl5/5.38 /usr/share/perl5 /
4-linux-gnu/perl/5.38 /usr/share/perl/5.38 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/
debconf: falling back to frontend: Teletype
Setting up libelf1t64:arm64 (0.190-1.1ubuntu0.1) ...
Setting up libmnl0:arm64 (1.0.5-2build1) ...
Setting up libxtables12:arm64 (1.8.10-3ubuntu2) ...
Setting up libbpf1:arm64 (1:1.3.0-2build2) ...
Setting up iproute2 (6.1.0-1ubuntu6.3) ...
debconf: unable to initialize frontend: Dialog
debconf: (No usable dialog-like program is installed, so the dialog based frontend cannot be used. at /usr
debconf: falling back to frontend: Readline
debconf: unable to initialize frontend: Readline
debconf: (Can't locate Term/ReadLine.pm in @INC (you may need to install the Term::ReadLine module) (@INC
inux-gnu/perl/5.38.2 /usr/local/share/perl/5.38.2 /usr/lib/aarch64-linux-gnu/perl5/5.38 /usr/share/perl5 /
4-linux-gnu/perl/5.38 /usr/share/perl/5.38 /usr/local/lib/site_perl) at /usr/share/perl5/Debconf/FrontEnd/
debconf: falling back to frontend: Teletype
Setting up nginx (1.24.0-2ubuntu7.12) ...
invoke-rc.d: could not determine current runlevel
invoke-rc.d: policy-rc.d denied execution of start.
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
root@05e51fe878ca:/#
```

Approach 2 - Verify Nginx

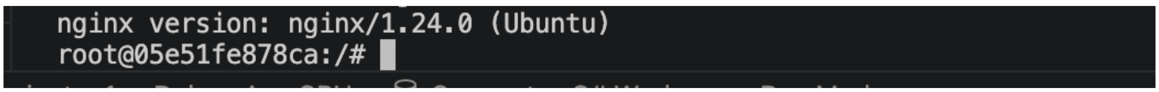
Command:

nginx -v

Explanation:

Displays the installed Nginx version.

Expected Output:



```
nginx version: nginx/1.24.0 (Ubuntu)
root@05e51fe878ca:/#
```

Approach 2 - Create Custom Page

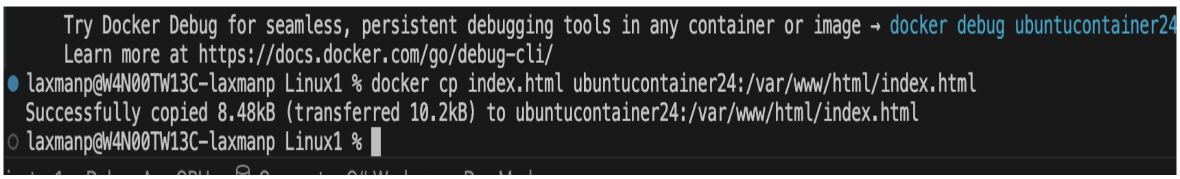
Command:

Edit /var/www/html/index.html

Explanation:

Creates a custom website page.

Expected Output:



Approach 2 - Start Nginx

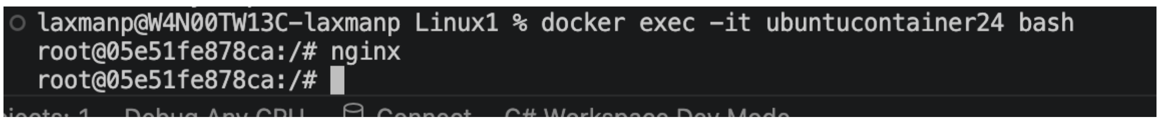
Command:

nginx

Explanation:

Starts the Nginx service.

Expected Output:



Approach 2 - Verify Locally

Command:

curl localhost

Explanation:

Requests the page from the web server.

Expected Output:

```
root@05e51fe878ca:/# nginx
root@05e51fe878ca:/# curl localhost
<!-- Sample HTML for demonstrating NGINX web server with Docker on Linux -->
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0"/>
    <title>Welcome - Our Landing Page</title>

    <!-- Internal CSS Styles -->
    <style>
        /* Reset and Base Styles */
        * {
            margin: 0;
            padding: 0;
            box-sizing: border-box;
        }

        body {
            font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
            line-height: 1.6;
            color: #333;
            background-color: linear-gradient(135deg, #667eea 0%, #764ba2 100%);
        }

        /* Navigation */
        nav {
            display: flex;
            justify-content: space-between;
            align-items: center;
            padding: 1.5rem 5%;
            background-color: rgba(255, 255, 255, 0.9);
```

Approach 2 - Commit Container

Command:

docker commit ubuntu-nginx ubuntu-nginx:v1

Explanation:

Saves the modified container as a reusable image.

Expected Output:

```
laxmanp@W4N00TW13C-laxmanp Linux1 % docker commit ubuntucontainer24 ubuntu-nginx:v1
sha256:309bbdd7d74f2ac9aa1615eb68f67a2d7a2acf073ef4409e3a29588c28a66f2c
laxmanp@W4N00TW13C-laxmanp Linux1 %
```

```
laxmanp@W4N00TW13C-laxmanp Linux1 % docker images

IMAGE                               ID             DISK USAGE   CONTENT SIZE   EXTRA
alpine:latest                       28bd5fe8b56d       13.6MB         4.27MB
compose-postgres-api-lab-api:latest 309a53ba5531        387MB           97MB   U
dpage/pgadmin4:latest               cefc4cc6b7d9        765MB          175MB   U
nginx:latest                        42f2d24ae18d        259MB         64.3MB   U
postgres:16                         081f1bc7bd5e        663MB          165MB   U
postgres:latest                     29ee7bb30d80        671MB          167MB   U
ubuntu-nginx:v1                     309bbdd7d74f        261MB         73.6MB
ubuntu:24.04                        786a8b558f7b        141MB         30.8MB   U
laxmanp@W4N00TW13C-laxmanp Linux1 %
```

Approach 2 - Run Published Website

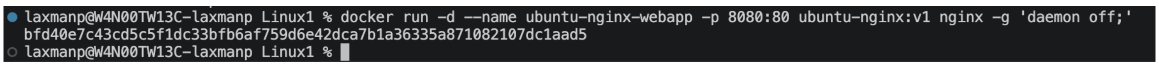
Command:

docker run -d --name ubuntu-nginx-web -p 8080:80 ubuntu-nginx:v1 nginx -g 'daemon off;'

Explanation:

Runs the custom image and exposes the website.

Expected Output:



```
laxmanp@W4N00TW13C-laxmanp Linux1 % docker run -d --name ubuntu-nginx-webapp -p 8080:80 ubuntu-nginx:v1 nginx -g 'daemon off;'
bfd40e7c43cd5c5f1dc33bfb6af759d6e42dca7b1a36335a871082107dc1aad5
laxmanp@W4N00TW13C-laxmanp Linux1 %
```